

Unity Programmer Task

Luciano Martin Franchino

For this technical task, I developed a 2D prototype. This decision was mainly based on efficiency and scope, as a 2D approach allowed me to focus on core systems such as inventory, interaction, and combat without adding unnecessary visual complexity. I also had prior experience with similar systems and suitable assets for this type of project.

The game presents a short, mission-based experience. The player wakes up in a forest and interacts with an NPC (a wizard) who assigns a task: defeat enemies (slimes), collect diamonds, and return them to complete the quest. Once the objective is fulfilled, the player receives a reward and the game ends with a victory screen. This structure allowed me to integrate multiple gameplay systems in a coherent and controlled manner.

From a technical perspective, the environment was built using tilesets with separate layers for ground and decorative elements. The player character features directional movement implemented with Unity's New Input System, including idle and movement animations. A health system with five life points is displayed through a UI slider, and upon reaching zero, a death animation plays before the level restarts.

Enemies are controlled by a single script using a simple state machine with idle, chase, and attack states. This approach makes the behavior easy to read and extend. Each enemy has one life point and drops a diamond upon defeat.

A core focus of the project is the inventory system. Items are defined using ScriptableObjects, allowing data to remain decoupled from scene logic. The Inventory Manager centralizes item collection, stacking, drag-and-drop behavior, equipment handling, and save data preparation. The inventory is slot-based, displayed in a grid, and supports item manipulation, deletion, and dynamic tooltips that show item icon, name, and description. The system also supports usable items (such as food, which restores health) and equipable items (such as the staff). Inventory persistence is handled through JSON serialization.

Combat is implemented through an equipable staff that fires projectiles. These projectiles are managed using an Object Pooling system to optimize performance and avoid constant instantiation and destruction. The overall architecture is modular, including dedicated managers for inventory and equipment, saving, dialogue, input, audio, weapon logic, and victory handling.

From a personal perspective, I greatly enjoyed working on this task. While the implementation required time for development and testing, the overall difficulty felt appropriate. Due to limited availability on the first day because of university commitments, I reused and adapted an inventory system from a previous project, extending it where necessary. Most systems were developed by me specifically for this task, with occasional use of AI tools to assist in structuring and debugging certain parts, particularly the save system and NPC interactions.

I am also aware that several systems could be further optimized and scaled by applying additional design patterns and architectural refinements. Given the time constraints of the task, my priority was to deliver a complete, functional, and stable prototype that clearly demonstrates my approach to problem-solving, system organization, and code quality. With more time, I would focus on improving extensibility and clean code.